INDIRA GANDHI NATIONAL OPEN UNIVERSITY
School of Computer and Information Sciences

MASTER OF COMPUTER APPLICATIONS (MCA_NEW / MCAOL)

Semester-2 Lab Assignment

MCSL-222 : OOAD and Web Technologies Lab

Assignment Number	MCA_NEW/MCAOL(II)/L-222/Assign/2026-27
Maximum Marks / Weightage	100 / 30%
Last date of submission	31st October, 2026

Name of Student	
Enrollment Number	
Study Centre Code	
Signature / Date	

SECTION 1 : OOAD LAB

Q1 (i) Class Diagram – Online Examination System [5 Marks]

Idea. Three kinds of users (Administrator, Examiner, Student) inherit from one class *User*. An Examiner creates Exams for a Course. An Exam has Questions. A Student attempts an Exam through an *ExamSession*, which stores the Answers and gives one Result.

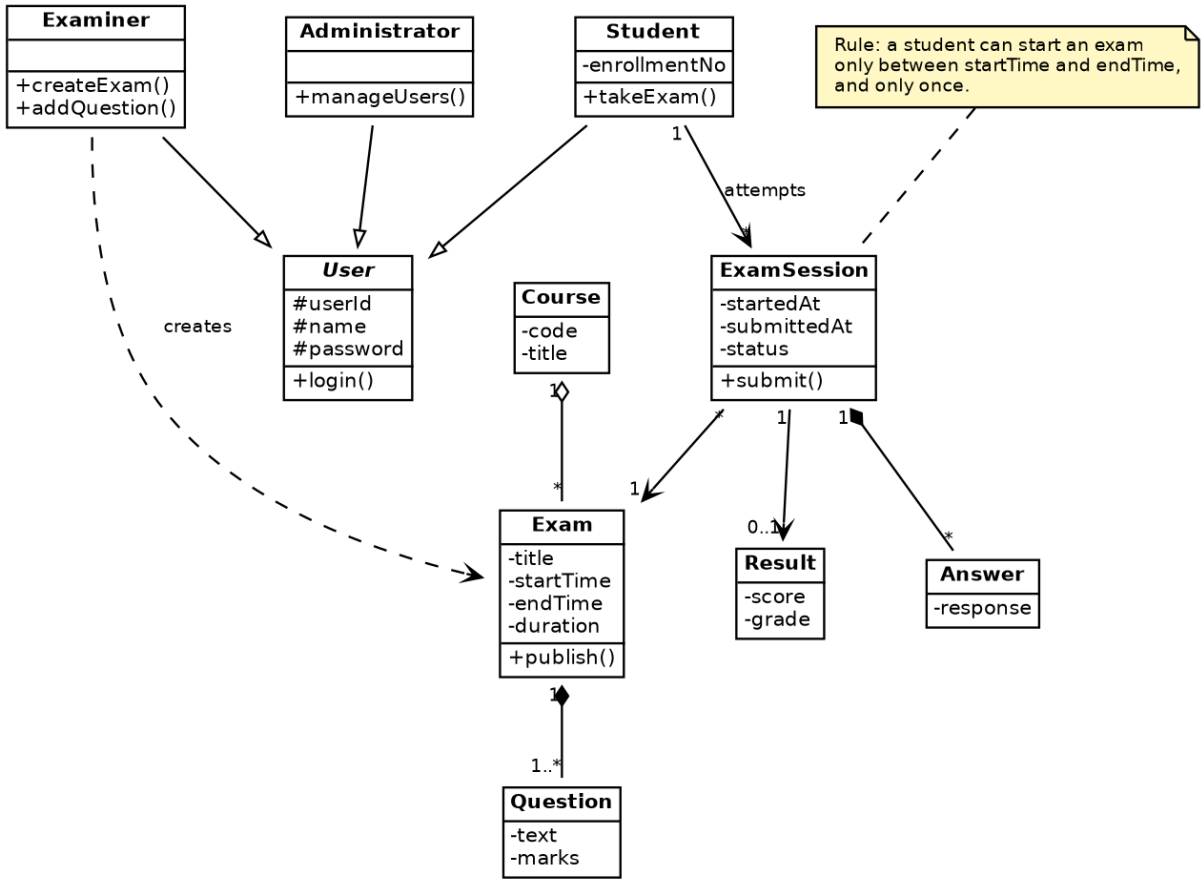


Figure 1.1 – Class diagram of the Online Examination System

Role	What the role is allowed to do (role-based boundary)
Administrator	Manage users and courses
Examiner	Create exams and questions for his own courses only
Student	Take an exam only between its start and end time, and only once; see only own result

Notation: hollow triangle = inheritance; filled diamond = composition (part cannot live without the whole); hollow diamond = aggregation; arrow = association with multiplicity (1, *, 0..1).

Run-time rule (note on diagram): an exam can be started only between *startTime* and *endTime*, and each student gets only one session per exam.

Q1 (ii) Deployment Diagram – Online Banking [5 Marks]

Idea. The system is split into zones. The Internet side is untrusted. The firewall and web server sit in the DMZ. The application server, database and backup are in a private network that cannot be reached directly from the Internet.

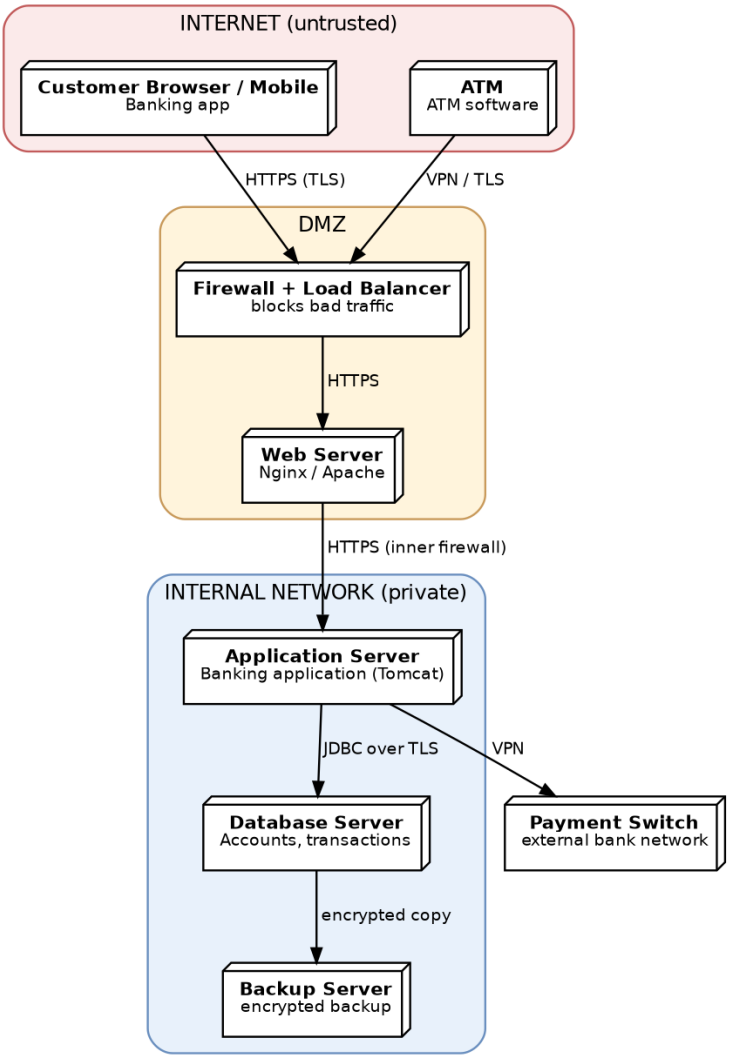


Figure 1.2 – Deployment diagram of the Online Banking infrastructure

Node	Job
Customer browser / mobile, ATM	Users reach the bank through HTTPS or VPN
Firewall + load balancer	Blocks bad traffic and shares load
Web server	Receives requests, forwards to the application
Application server	Runs banking logic (login, transfer, balance)
Database server	Stores accounts and transactions; allows connections only from the application server
Backup server	Keeps encrypted copy of data
Payment switch	Connects to other banks (VPN)

Q1 (iii) State Machine Diagram – Re-registration and Fee Payment (IGNOU MCA) [5 Marks]

Idea. A learner logs in, the system checks eligibility, the learner pays the fee and gets registered. Labels on arrows are written as *event [guard] / action*; a guard is a condition that must be true for the arrow to be taken.

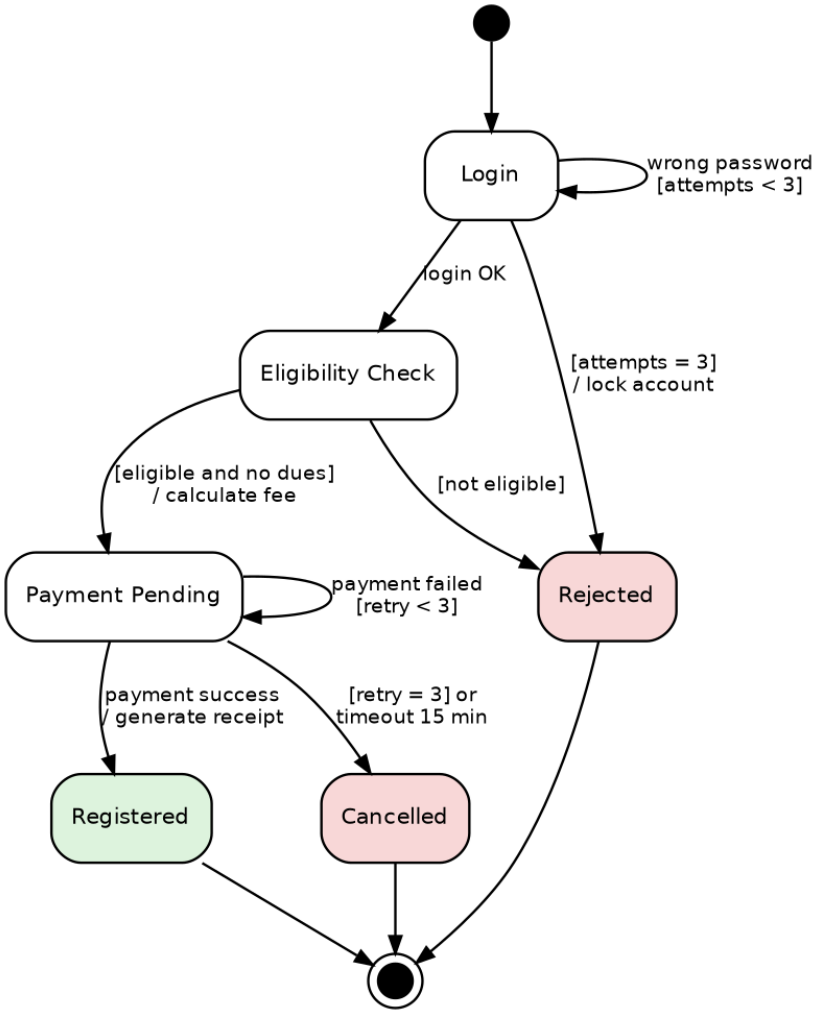


Figure 1.3 – State machine of Online Re-registration and Fee Payment

State	Meaning
Login	Learner enters enrollment number and password. After 3 wrong attempts the account is locked
Eligibility Check	Programme active, re-registration window open, no pending dues
Payment Pending	Fee is calculated and payment is done through the gateway; up to 3 retries, 15 minute time-out
Registered	Payment successful, receipt is generated (final state)
Rejected / Cancelled	Not eligible, account locked, or payment not completed (final states)

Q1 (iv) Sequence Diagram – ATM System [5 Marks]

Idea. Time flows from top to bottom. Solid arrow with filled head = synchronous call (sender waits). Solid arrow with open head = asynchronous message. Dashed arrow = return. Boxes show conditions: **loop** (PIN retries), **alt** (if / else), **opt** (exception: card retained after 3 wrong PINs).

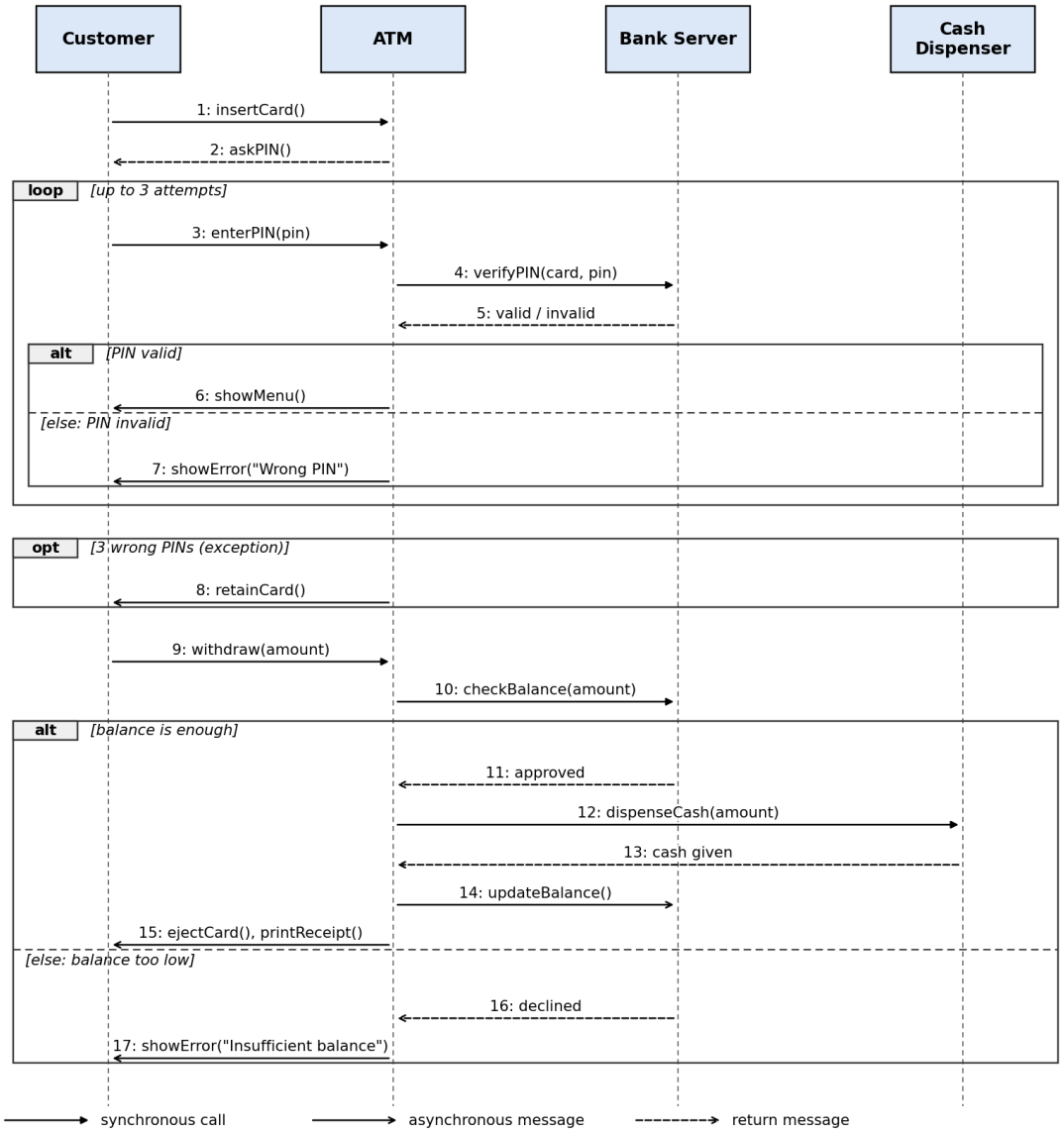


Figure 1.4 – Sequence diagram of ATM cash withdrawal

SECTION 2 : WEB TECHNOLOGIES LAB

Q1 (i) JSP + JDBC – Student Profile Update [10 Marks]

Assumptions

- Tomcat 9, JDK 11, MySQL 8; MySQL Connector/J jar copied into *WEB-INF/lib*.
- Database *ignou* with one table *student* (schema below). Connection: *localhost:3306*, user *root*, password *root* (change as needed).
- One input box accepts either the Enrollment Number or the Mobile Number; the SQL checks both columns.
- The student can change address, mobile and email. Name and enrollment number cannot be changed.
- To keep it simple, the lookup uses only the number (no password/OTP). A real system would also verify with OTP.

Database schema

schema.sql

```
CREATE DATABASE ignou;
USE ignou;

CREATE TABLE student (
  enrollment_no VARCHAR(10)  PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  mobile        VARCHAR(10)  NOT NULL UNIQUE,
  email         VARCHAR(100) NOT NULL,
  address       VARCHAR(255) NOT NULL
);

INSERT INTO student VALUES
 ('2512345678', 'Rahul Sharma', '9876543210', 'rahul@example.com', 'H.No 12, Sector 5, Dwarka, New Delhi'),
 ('251234567',  'Priya Verma',  '9123456780', 'priya@example.com', '45 MG Road, Pune');
```

Project files

Layout

```
StudentProfile/
 ├── index.jsp       (search box)
 ├── profile.jsp     (find student, show edit form)
 ├── update.jsp      (validate and save)
 └── WEB-INF/
      ├── lib/mysql-connector-j.jar
      └── classes/util/DB.java    (database connection)
```

WEB-INF/classes/util/DB.java

```
package util;

import java.sql.*;

public class DB {
    // Connection settings (assumption: MySQL on localhost)
    private static final String URL  = "jdbc:mysql://localhost:3306/ignou";
    private static final String USER = "root";
    private static final String PASS = "root";

    public static Connection getConnection() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

index.jsp

```
<html>
<body>
  <h2>Student Profile Update</h2>
  <form action="profile.jsp" method="post">
    Enrollment No. or Mobile No.:
    <input type="text" name="token" required>
    <input type="submit" value="Search">
  </form>
</body>
</html>
```

profile.jsp

```
<%@ page import="java.sql.*, util.DB" %>
<%!
  // escape text before printing it in HTML (stops script injection)
  String esc(String s) {
    return s == null ? "" : s.replace("&", "&amp;").replace("<", "&lt;")
                             .replace(">", "&gt;").replace("\"", "&quot;");
  }
%>
<html>
<body>
<%
  String token = request.getParameter("token");
  try (Connection con = DB.getConnection();
       PreparedStatement ps = con.prepareStatement(
         "SELECT * FROM student WHERE enrollment_no = ? OR mobile = ?")) {
    ps.setString(1, token);
    ps.setString(2, token);
    ResultSet rs = ps.executeQuery();
    if (rs.next()) {
%>
  <h2>Welcome, <%= esc(rs.getString("name")) %></h2>
  <form action="update.jsp" method="post">
    <input type="hidden" name="enrollment_no" value="<%= esc(rs.getString("enrollment_no")) %>">
    Enrollment No: <b><%= esc(rs.getString("enrollment_no")) %></b><br><br>
    Address: <input type="text" name="address" size="50" value="<%= esc(rs.getString("address")) %>"><br><br>
    Mobile:  <input type="text" name="mobile" value="<%= esc(rs.getString("mobile")) %>"><br><br>
    Email:   <input type="text" name="email" value="<%= esc(rs.getString("email")) %>"><br><br>
    <input type="submit" value="Update">
  </form>
<%
    } else {
      out.println("<p style='color:red'>Student not found.</p><a href='index.jsp'>Back</a>");
    }
  } catch (Exception e) {
    out.println("<p style='color:red'>Error: could not read data.</p>");
  }
%>
</body>
</html>
```

update.jsp

```
<%@ page import="java.sql.*, util.DB" %>
<html>
<body>
<%
  String enrollment = request.getParameter("enrollment_no");
  String address    = request.getParameter("address");
  String mobile     = request.getParameter("mobile");
  String email      = request.getParameter("email");

  // simple validation
  if (address == null || address.trim().length() < 5) {
    out.println("Address is too short.");
  } else if (mobile == null || !mobile.matches("[6-9][0-9]{9}")) {
    out.println("Mobile must be a valid 10 digit number.");
  } else if (email == null || !email.matches("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}")) {
    out.println("Email is not valid.");
  } else {
    try (Connection con = DB.getConnection();
         PreparedStatement ps = con.prepareStatement(
           "UPDATE student SET address = ?, mobile = ?, email = ? WHERE enrollment_no = ?")) {
      ps.setString(1, address.trim());
      ps.setString(2, mobile);
      ps.setString(3, email);
      ps.setString(4, enrollment);
      int rows = ps.executeUpdate();
      out.println(rows == 1 ? "Profile updated successfully." : "Update failed.");
    } catch (SQLIntegrityConstraintViolationException e) {
      out.println("This mobile number is already used by another student.");
    } catch (Exception e) {
      out.println("Error: update failed.");
    }
  }
%>
<br><a href="index.jsp">Back</a>
</body>
</html>
```

How it works

- **index.jsp**: student types enrollment or mobile number and clicks Search.
- **profile.jsp**: runs *SELECT … WHERE enrollment_no = ? OR mobile = ?*. If found, shows the editable form; otherwise “Student not found”.

- **update.jsp**: validates address, mobile (10 digits starting 6-9) and email, then runs *UPDATE* using the enrollment number.
- **Safety:** PreparedStatement (stops SQL injection), *esc()* function (stops script injection), server-side validation.

Sample input and output

Input	Output
Number 2512345678	Page “Welcome, Rahul Sharma” with address, mobile and email boxes
Number 9876543210 (mobile)	Same student is shown
Number 1111111111	Student not found.
Change address to “Flat 8, Green Park, Dwarka”, click Update	Profile updated successfully.
Mobile = 12345	Mobile must be a valid 10 digit number.
Mobile of another student (9123456780)	This mobile number is already used by another student.

Run the project in Tomcat and attach your own screenshots of these cases.

Q1 (ii) Spring Boot + Spring Security + JPA – Workshop Registration and Fee Payment [10 Marks]

Assumptions

- Java 11, Maven, Spring Boot 2.7, Spring Data JPA (Hibernate), Spring Security, H2 in-memory database (can be changed to MySQL in *application.properties*).
- Layers: Controller (REST API) → Repository (JPA) → Database.
- Two roles: **USER** (participant) and **ADMIN**. Demo users: *admin/admin123*, *student1/pass123*, *student2/pass123* (passwords stored as BCrypt hashes).
- Workshop fee is fixed at 1500 on the server. Payment is simulated by the */pay* call; a real portal would use a payment gateway.
- Payment status is PENDING or PAID. **Once PAID, the registration cannot be updated or deleted.**

Rules

Action	Who	PENDING	PAID
Create (POST)	USER, ADMIN	Allowed	–
Read own (GET)	USER, ADMIN	Allowed	Allowed
Read all (GET /all)	ADMIN only	Allowed	Allowed
Update (PUT)	Owner or ADMIN	Allowed	Blocked – 409
Delete (DELETE)	Owner or ADMIN	Allowed	Blocked – 409
Pay (POST /pay)	Owner or ADMIN	Allowed	Blocked – 409 “Already paid”

A USER who asks for another user’s registration gets **404 Not found**.

Source code

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.18</version>
  </parent>
  <groupId>edu.ignou</groupId>
  <artifactId>workshop</artifactId>
  <version>1.0</version>
  <properties><java.version>11</java.version></properties>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        web</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
        jpa</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        security</artifactId></dependency>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-
        validation</artifactId></dependency>
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><scope>runtime</scope></dependency>
  </dependencies>
  <build><plugins><plugin>
    <groupId>org.springframework.boot</groupId><artifactId>spring-boot-maven-plugin</artifactId>
  </plugin></plugins></build>
</project>
```

src/main/resources/application.properties

```
spring.datasource.url=jdbc:h2:mem:workshopdb
spring.jpa.hibernate.ddl-auto=update
server.error.include-message=always
```

App.java

```
package edu.ignou.workshop;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }
}
```

Registration.java (Entity)

```
package edu.ignou.workshop;

import javax.persistence.*;

@Entity
public class Registration {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String owner;            // username of the participant who registered
    private double amount;           // workshop fee
    private String paymentStatus;    // PENDING or PAID

    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getOwner() { return owner; }
    public void setOwner(String owner) { this.owner = owner; }
    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }
    public String getPaymentStatus() { return paymentStatus; }
    public void setPaymentStatus(String s) { this.paymentStatus = s; }
}
```

RegistrationRepository.java

```
package edu.ignou.workshop;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface RegistrationRepository extends JpaRepository<Registration, Long> {
    List<Registration> findByOwner(String owner);
}
```

RegistrationController.java (CRUD + payment rules)

```
package edu.ignou.workshop;

import java.util.List;
import javax.validation.Valid;
import javax.validation.constraints.*;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.server.ResponseStatusException;

@RestController
@RequestMapping("/api/registrations")
public class RegistrationController {

    private final RegistrationRepository repo;
    public RegistrationController(RegistrationRepository repo) { this.repo = repo; }

    // data sent by the client (no id / owner / status, so the client cannot fake them)
    static class Form {
        @NotBlank public String name;
        @NotBlank @Email public String email;
    }

    // CREATE
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
    public Registration create(@Valid @RequestBody Form f, Authentication auth) {
        Registration r = new Registration();
        r.setName(f.name);
        r.setEmail(f.email);
        r.setOwner(auth.getName());
        r.setAmount(1500);
        r.setPaymentStatus("PENDING");
        return repo.save(r);
    }

    // READ: my registrations
    @GetMapping
    public List<Registration> mine(Authentication auth) { return repo.findByOwner(auth.getName()); }

    // READ: all registrations (ADMIN only, see SecurityConfig)

    @GetMapping("/all")
    public List<Registration> all() { return repo.findAll(); }

    @GetMapping("/{id}")
    public Registration one(@PathVariable Long id, Authentication auth) { return find(id, auth); }
```

```
    // UPDATE: not allowed after payment
    @PutMapping("/{id}")
    public Registration update(@PathVariable Long id, @Valid @RequestBody Form f, Authentication auth) {
        Registration r = find(id, auth);
        if (r.getPaymentStatus().equals("PAID"))
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Paid registration cannot be changed");
        r.setName(f.name);
        r.setEmail(f.email);
        return repo.save(r);
    }

    // DELETE: not allowed after payment
    @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id, Authentication auth) {
        Registration r = find(id, auth);
        if (r.getPaymentStatus().equals("PAID"))
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Paid registration cannot be deleted");
        repo.delete(r);
    }

    // PAY: marks the fee as paid (a real portal would call a payment gateway here)
    @PostMapping("/{id}/pay")
    public Registration pay(@PathVariable Long id, Authentication auth) {
        Registration r = find(id, auth);
        if (r.getPaymentStatus().equals("PAID"))
            throw new ResponseStatusException(HttpStatus.CONFLICT, "Already paid");
        r.setPaymentStatus("PAID");
        return repo.save(r);
    }

    // find a row; a normal user can only see his own rows, an admin can see all
    private Registration find(Long id, Authentication auth) {
        Registration r = repo.findById(id)
            .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Not found"));

        boolean admin = auth.getAuthorities().stream()
            .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN"));
        if (!admin && !r.getOwner().equals(auth.getName()))
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "Not found");
        return r;
    }
}
```

SecurityConfig.java (Authentication + RBAC)

```
package edu.ignou.workshop;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    // Authentication: three demo users (passwords stored as BCrypt hashes)
    @Bean
    UserDetailsService users(PasswordEncoder enc) {
        UserDetails admin = User.withUsername("admin").password(enc.encode("admin123")).roles("ADMIN").build();
        UserDetails s1 = User.withUsername("student1").password(enc.encode("pass123")).roles("USER").build();
        UserDetails s2 = User.withUsername("student2").password(enc.encode("pass123")).roles("USER").build();
        return new InMemoryUserDetailsManager(admin, s1, s2);
    }

    // Authorization (RBAC)
    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(c -> c.disable())     // REST API with HTTP Basic, no cookies
            .authorizeRequests(a -> a
                .antMatchers(HttpMethod.GET, "/api/registrations/all").hasRole("ADMIN")
                .antMatchers("/api/registrations/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().denyAll())
            .httpBasic(Customizer.withDefaults());
        return http.build();

    }
}
```

Run

Commands

```
mvn spring-boot:run          # server starts on http://localhost:8080
```

Sample input and output

Test run (curl)

```
# 1. student1 registers
$ curl -u student1:pass123 -X POST localhost:8080/api/registrations -H "Content-Type: application/json" \
       -d '{"name":"Rahul Sharma","email":"rahul@example.com"}'
201  {"id":1,"name":"Rahul
    Sharma","email":"rahul@example.com","owner":"student1","amount":1500.0,"paymentStatus":"PENDING"}

# 2. update while PENDING -> allowed
$ curl -u student1:pass123 -X PUT localhost:8080/api/registrations/1 -H "Content-Type: application/json" \
       -d '{"name":"Rahul K Sharma","email":"rahul@example.com"}'
200  {"id":1,"name":"Rahul K Sharma", ... "paymentStatus":"PENDING"}

# 3. pay
$ curl -u student1:pass123 -X POST localhost:8080/api/registrations/1/pay
200  {"id":1, ... "paymentStatus":"PAID"}

# 4. update or delete after payment -> blocked
$ curl -u student1:pass123 -X DELETE localhost:8080/api/registrations/1
409  "Paid registration cannot be deleted"

# 5. another student tries to read it
$ curl -u student2:pass123 localhost:8080/api/registrations/1
404  "Not found"

# 6. student tries admin URL
$ curl -u student1:pass123 localhost:8080/api/registrations/all
403  Forbidden

# 7. admin sees all
$ curl -u admin:admin123 localhost:8080/api/registrations/all
200  [ {"id":1, ... "owner":"student1","paymentStatus":"PAID"} ]

# 8. wrong data / no login
$ curl -u student1:pass123 -X POST localhost:8080/api/registrations -H "Content-Type: application/json" -d
    '{"name":"","email":"abc"}'
400  (validation error)
$ curl localhost:8080/api/registrations
401  (login required)
```

Run the project and attach your own screenshots (Postman or curl) of these cases.